

Backend Ledger Project: Deep Interview Questions

Target Repo: backend-ledger | **Modules Covered:** user.model.js, account.model.js, ledger.model.js, transaction.controller.js, auth.middleware.js

This PDF contains 25 highly specific, technical interview questions directly based on the implementation details, Mongoose schemas, controllers, and edge cases of this backend-ledger repository. Interviewers ask these exact questions to evaluate whether you truly built and understand your project.

Module 1: Architecture, Schemas & Ledger Design

Q1. Why is there no 'balance' field in your accountSchema (account.model.js)? How is balance calculated?

Answer: Balance is not stored as a mutable number to prevent race conditions and data corruption. Instead, `accountModel.methods.getBalance()` calculates balance dynamically using a MongoDB Aggregation Pipeline: it filters entries for the account (`$match`), sums all DEBIT and CREDIT entries (`$group` with `$cond`), and subtracts total DEBIT from total CREDIT (`$project`).

Q2. How is your Ledger schema (ledger.model.js) enforced to be strictly Immutable?

Answer: Immutability is enforced at two levels:

1. **Schema Definition:** Fields `account`, `amount`, `transaction`, and `type` are marked with `immutable: true`.
2. **Mongoose Pre-Hooks:** Middleware hooks (`pre('findOneAndUpdate')`, `pre('updateOne')`, `pre('deleteOne')`, etc.) call `preventLedgerModification()` which throws an explicit Error preventing any updates or deletions.

Q3. What is the purpose of the 'systemUser' field in userSchema and authSystemUserMiddleware?

Answer: The `systemUser` boolean field (marked `select: false` and `immutable: true`) identifies system administrative accounts. The `authSystemUserMiddleware` verifies this flag to grant access to system-only endpoints like `createInitialFundsTransaction`, allowing the system to mint/seed initial funds into user accounts.

Q4. How did you implement JWT Logout and Token Revocation in this project?

Answer: Via `tokenBlackListModel`

(`[blackList.model.js](file:///c:/Users/shrad/backend-ledger/src/models/blackList.model.js)`). On logout, the token is extracted from cookies/headers and inserted into the blacklist collection. `authMiddleware` queries this collection before verifying JWT. The collection has a MongoDB TTL index (`expireAfterSeconds: 259200` i.e. 3 days) to auto-delete expired tokens.

Q5. How does transaction.controller.js enforce Idempotency?

Answer: The `transactionSchema` requires an `idempotencyKey` with a `unique: true` index. Before initiating a transfer, `createTransaction` checks if a transaction with that key already exists. If status is `COMPLETED`, it immediately returns HTTP 200 with the processed transaction instead of re-executing.

Q6. Why did you use MongoDB Sessions (startSession()) in createTransaction?

Answer: A single transfer requires writing to both the transactions collection (status update) and creating TWO records in the ledgers collection (DEBIT & CREDIT). Multi-document transactions via mongoose.startSession() ensure ACID compliance — all 3 documents are written atomically or none are.

Module 2: Code Walkthrough & Deep Implementation Questions

Q7. Walk me through lines 32-64 in account.model.js (the getBalance function).

Answer:

- \$match: Filters ledger entries matching this._id.
- \$group: Groups all matching records and uses \$cond to evaluate if type === 'DEBIT' (sums into totalDebit) or CREDIT (sums into totalCredit).
- \$project: Subtracts totalDebit from totalCredit to return the net balance.

Q8. In auth.controller.js, why is .select('+password') used during login?

Answer: In userSchema, password has select: false to prevent leaking password hashes in standard user queries (e.g. GET endpoints). During login, Mongoose excludes password by default, so .select('+password') explicitly includes it so bcrypt.compare() can run.

Q9. How does the password hashing pre-save hook work in user.model.js?

Answer: Before saving a user document, the pre('save') hook checks this.isModified('password'). If modified (or new), it hashes the plain text password using bcrypt.hash(this.password, 10) before persisting.

Q10. How does auth.middleware.js extract JWT tokens from requests?

Answer: It checks two locations: req.cookies.token (for cookie-based web clients) OR req.headers.authorization?.split(' ')[1] (for Bearer tokens from mobile/API clients). If neither exists, it returns HTTP 401 Unauthorized.

Q11. What is the difference between createTransaction and createInitialFundsTransaction?

Answer: createTransaction is for peer-to-peer user transfers (requires checking sender balance). createInitialFundsTransaction is executed by a systemUser to seed money into a target account from a system master account.

Q12. Why are fromAccount and toAccount status checked before processing?

Answer: Both accounts must have status ACTIVE. If either account is FROZEN or CLOSED, the transfer is rejected (HTTP 400), preventing transfers to suspended or terminated accounts.

Module 3: Code Review, Flaws & Spotting Bugs (Interview Favorites)

Q13. CRITIQUE: Spot the bug in transaction.controller.js lines 151-157 (catch block).

Answer: In createTransaction, if an error occurs inside the try block, the catch block returns HTTP 400, but ****fails to call session.abortTransaction() and session.endSession()****! This leaks MongoDB session resources and leaves locks hanging in RAM.

Q14. CRITIQUE: What is the issue with line 131 in transaction.controller.js?

Answer: Line 131 introduces an artificial 15-second delay (setTimeout 15*1000) inside an active database transaction session. Holding open transactions for 15 seconds starves the Mongoose connection pool under concurrent load.

Q15. CRITIQUE: Does createTransaction handle Race Conditions / Double Debits?

Answer: No! In the current controller, getBalance() is called at step 4 ***outside the transaction session***. If two concurrent requests hit at the exact same millisecond, both read balance as █100 before either inserts a DEBIT entry, leading to double-spending (-█100 balance).

Q16. CRITIQUE: Is your email notification blocking the HTTP response?

Answer: Yes. await emailService.sendTransactionEmail(...) is called synchronously at step 10 before returning HTTP 201. If the email server is slow or fails, the API response is delayed or crashes. Fix: Queue emails asynchronously.

Q17. CRITIQUE: What happens if an Idempotency Key is reused with a different amount?

Answer: Currently, the code only checks if idempotencyKey exists; it does not verify if the payload matches! A malicious client could send an old key with a new █50,000 amount and receive the old cached response. Fix: Hash the request payload alongside the idempotency key.

Q18. CRITIQUE: Is there a Self-Transfer check in createTransaction?

Answer: Currently missing! A user can pass fromAccount === toAccount, which creates duplicate DEBIT and CREDIT entries on the same account. Fix: Add if (fromAccount === toAccount) return res.status(400).

Module 4: System Design, Scaling & Production Readiness

Q19. How would you optimize getBalance() as ledger entries reach millions?

Answer: Summing millions of rows per request will cause slow DB queries. Fix:

1. Ensure compound index `{ account: 1, type: 1 }` on `ledgerSchema`.
2. Implement **Periodic Balance Snapshotting** (e.g. daily balances saved in Account doc). `getBalance()` then sums only ledger entries created after the last snapshot timestamp.

Q20. How would you scale this Node.js backend to 10 instances behind Nginx?

Answer:

1. Ensure stateless session auth (JWT).
2. Move Token Blacklist from MongoDB to Redis for sub-millisecond in-memory lookups.
3. Use Redis distributed locks (`Redlock`) across instances during balance checks.

Q21. Why is Redis better than MongoDB for Token Blacklisting under high traffic?

Answer: `authMiddleware` runs on every single protected API request. Querying MongoDB per request adds disk I/O latency. Redis runs in RAM (sub-millisecond) and supports native key expiration (TTL) without requiring background Mongoose cleanups.

Q22. How would you implement Multi-Currency support (INR, USD, EUR)?

Answer: Store currency on `accountSchema` (already present). When transferring between different currencies, fetch real-time FX rates, convert amount before debiting/crediting, and record the `exchangeRate` snapshot on `transactionSchema`.

Q23. How would you implement Transaction Reversals / Refunds?

Answer: Because ledger entries are immutable (`preventLedgerModification`), you CANNOT delete or modify existing records. You must append a NEW `REVERSAL` transaction that creates swapped DEBIT and CREDIT entries for the accounts.

Q24. How does MongoDB Replica Set failover affect active transaction sessions?

Answer: If a primary node fails mid-transaction, Mongoose throws a `TransientTransactionError`. Production code must wrap transaction logic in an automatic retry loop (up to 3 retries).

Q25. How would you write an Integration Test for createTransaction in Supertest?

Answer: Use `supertest` with `mongodb-memory-server`. 1. Register and log in 2 users. 2. Mint initial funds to User A. 3. Call POST `/api/transactions` with payload. 4. Assert HTTP status 201, verify `transactions` status is `COMPLETED`, and assert `getBalance()` for User A decreased by amount while User B increased.